Libraries

2026 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 5
2.0.0

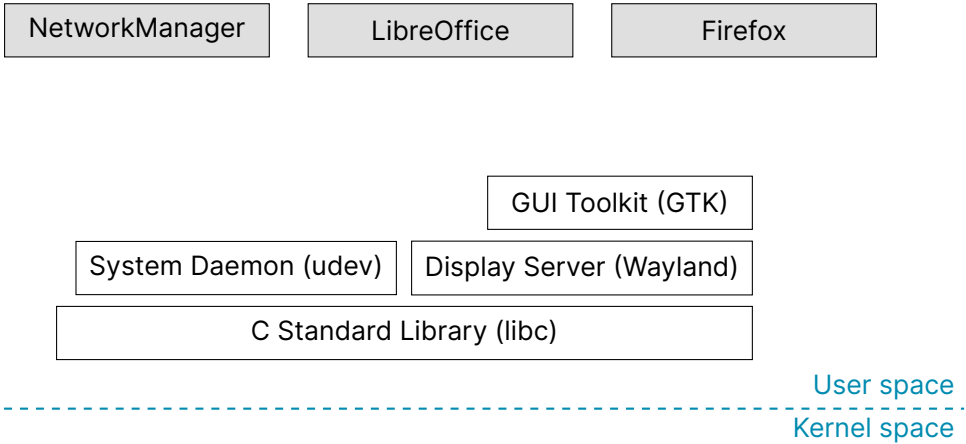
Based on original slides by Professor Jon Eyolfson.

What is an Operating System?

The kernel is part of an operating system, but what else?

Are macOS, iOS, iPadOS, watchOS, and tvOS all different operating systems?

Applications May Pass Through Multiple Layers of Libraries



What an Operating System is Depends on the Application

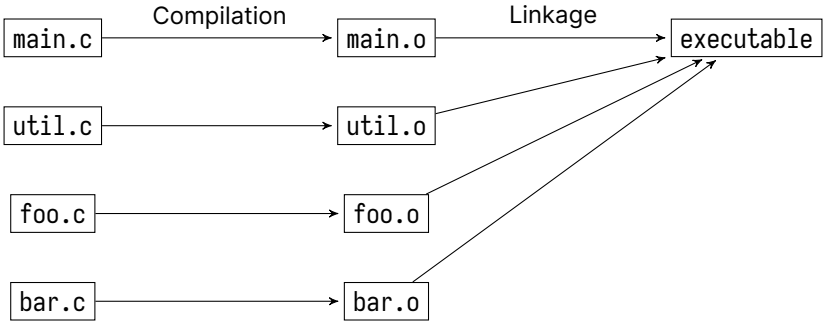
Android and Debian both use the Linux kernel,
but the applications are different

Maybe they're the same OS if you only care about terminal applications

"Linux" distributions may be considered GNU/Linux
GNU distributes the standard C library and common utilities

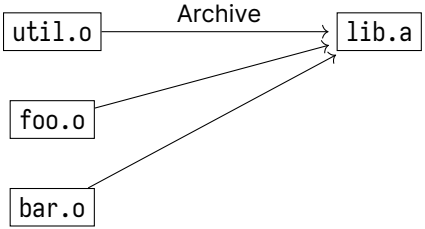
An OS consists of a kernel and libraries required for your application

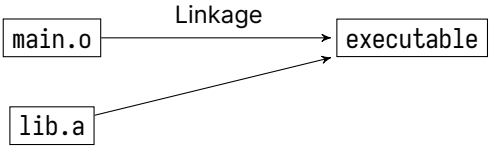
Normal Compilation in C



Note: object files (`.o`) are just ELF files with code for functions

Static Libraries Are Included At Link Time

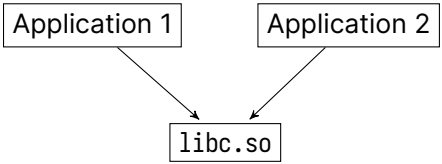




Dynamic Libraries Are For Reusable Code

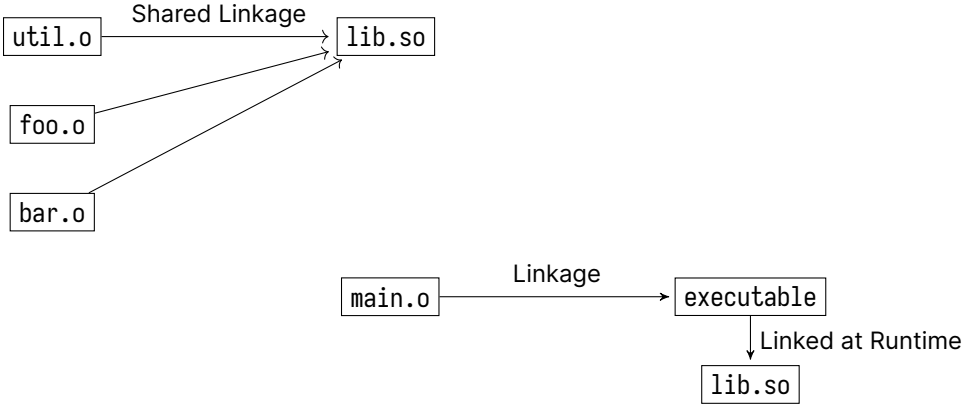
The C standard library is a dynamic library (`.so`), like any other on the system
Basically a collection of `.o` files containing function definitions

Multiple applications can use the same library:



The operating system only loads `libc.so` in memory once, and shares it

Dynamic Libraries Are Included At Runtime



Useful Command Line Utility for Dynamic Libraries

`ldd <executable>`

shows which dynamic libraries an executable uses

Static vs Dynamic Libraries

To statically link your code
Basically copies the `.o` files directly into the executable

The drawbacks compared to dynamic libraries:

- Statically linking prevents re-using libraries
  (commonly used libraries have many duplicates)
- Any updates to a static library requires the executable to be recompiled

What are issues with dynamic libraries?

Dynamic Libraries Updates Can Break Executables

A dynamic library update may subtly change the ABI causing a crash

Consider the following in a dynamic library:
A `struct` with multiple fields corresponding to a specific data layout (C ABI)

An executable accesses the fields of the `struct` used by a dynamic library

Now if a dynamic library reorders the fields
The executable uses the old offsets and is now wrong

Note: this is OK if the dynamic library never exposes the fields of a `struct`

C Uses a Consistent ABI for `structs`

`structs` are laid out in memory with the fields matching the declaration order
- C compilers ensure the ABI of `structs` are the consistent for an architecture

Consider the following structures:

Library v1:

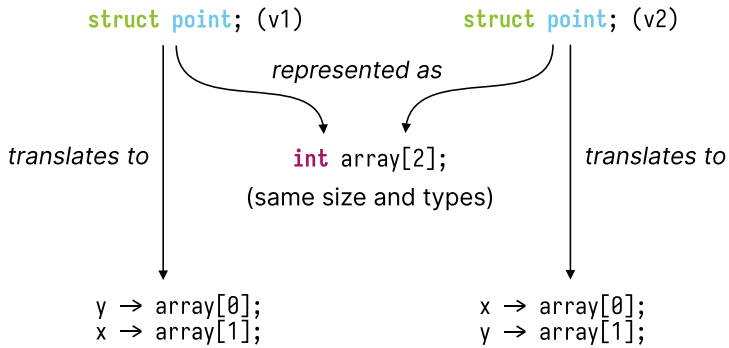
```
struct point {
  int y;
  int x;
};
```

Library v2:

```
struct point {
  int x;
  int y;
};
```

For v1 the x field is offset by 4 bytes from the start of `struct point`'s base
- For v2 it is offset by 0 bytes, and this difference will cause problems

After Compilation the Translation Differs for Each Version



The Point API Has Four Functions

libpoint.so

```
struct point *point_create(int x, int y);
int point_get_x(struct point *p);
int point_get_y(struct point *p);
void point_destroy(struct point *p);
```

ABI Stable Code Should Always Print “1, 2” for Both Lines

```c
#include <stdio.h>
#include <stdlib.h>

#include "point.h"

int main(void) {
  struct point *p = point_create(1, 2);

  printf("point (x, y) = %d, %d (using library)\n",
         point_get_x(p), point_get_y(p));

  printf("point (x, y) = %d, %d (using struct)\n", p->x, p->y);

  point_destroy(p);
  return 0;
}
```

Mismatched Versions Don't Agree on the Location of X and Y